Defending the Peg: Real-Time Dynamic Protection and Anomaly Detection in DeFi Stablecoins

HENGXING ZENG*, Hainan University, China
SHIPENG YE*, Hainan University, China
XIAOQI LI, Hainan University, China

With the rapid evolution of the Decentralized Finance (DeFi) ecosystem, stablecoins have emerged as a critical infrastructure bridging the cryptocurrency market with traditional financial paradigms. However, stablecoin systems rely heavily on smart contracts to execute automated operations. The immutable nature of these systems post-deployment means that the exploitation of security vulnerabilities can lead to irreversible, massive economic losses and potentially trigger systemic financial risks. Current research on stablecoin smart contract security faces challenges such as a lack of domain-specific targeting and the obsolescence of static defense models. To address this, this paper systematically analyzes common attack vectors in stablecoin environments and proposes a practical, real-time dynamic defense architecture. By analyzing 12 real-world security incidents, we elucidate the underlying mechanisms of high-risk patterns such as reentrancy attacks, oracle manipulation, and composite flash loan attacks. Concurrently, we construct a real-time anomaly detection model utilizing multi-dimensional on-chain temporal features and the Bi-LSTM algorithm. Experimental results demonstrate that this model achieves a classification accuracy of 96.61%, with an average recall rate of 97.70% for malicious attack samples, and a single inference latency ranging from 1.5 to 2.8 milliseconds.

Additional Key Words and Phrases: Stablecoin, Smart Contract, Security Attacks, Dynamic Protection, Anomaly Detection

1 Introduction

With the continuous evolution of blockchain technology, the novel financial paradigm represented by Decentralized Finance has rapidly emerged, dismantling the geographical and institutional barriers of traditional finance and providing intermediary-free financial services globally [3, 26, 53]. As a core infrastructure of the DeFi ecosystem, stablecoins maintain value stability by pegging to fiat currencies or other assets. They effectively resolve the extreme volatility inherent in crypto assets, playing a crucial role as a medium of exchange, a store of value, and a form of collateral.

Recent market data indicates that by 2025, the total market capitalization of global stablecoins exceeded $260 billion, accounting for over 70% of the daily trading volume in the cryptocurrency market and serving as the primary source of liquidity [57]. Stablecoin systems rely heavily on smart contracts to automate operations such as token minting, burning, collateral management, and liquidation without manual intervention. However, the post-deployment immutability of smart contracts dictates that exploited vulnerabilities can cause irreversible and catastrophic economic losses [31]. Historically, smart contract vulnerabilities have constituted a primary risk factor in the DeFi ecosystem. For instance, the infamous 2016 DAO attack, caused by a reentrancy vulnerability, led to the Ethereum hard fork [27, 34, 40]. Subsequent incidents, including the 2017 Parity multisig wallet exploit [13], the 2018 Beauty Chain (BEC) overflow bug [46], and the $50 million exploit on the Binance Smart Chain (BSC) during a contract migration in 2021 [8], profoundly underscore the critical urgency of stablecoin smart contract security[36].

Regarding specific vulnerability taxonomies, stablecoin smart contracts frequently encounter security flaws such as reentrancy attacks, integer overflows, access control defects, and oracle manipulation. Reentrancy attacks exploit external call mechanisms to repeatedly execute critical

*These authors contributed equally to this work.

Authors' Contact Information: Hengxing Zeng, Hainan University, Haikou, China, 1434780680@qq.com; Shipeng Ye, Hainan University, Haikou, China, yeshipeng35@gmail.com; Xiaoqi Li, Hainan University, Haikou, China, csxqli@ieee.org.

logic [25, 34, 35]. Integer overflows cause anomalous asset calculations and illegal token issuance [20]. Access control defects grant attackers unauthorized privileges to hijack core system functions [2]. Furthermore, oracle manipulation alters external price feeds to indirectly influence system behavior, serving as a primary risk for crypto-collateralized stablecoins. More alarmingly, these vulnerabilities are increasingly combined with novel financial instruments, such as flash loans, to orchestrate composite attacks [52]. In a single, atomic, and risk-free transaction, attackers can complete capital acquisition, market manipulation, arbitrage, and loan repayment [14]. This atomic execution significantly amplifies both the efficiency and stealth of the attacks, posing profound challenges to conventional security paradigms[11, 54].

Moreover, different categories of stablecoins exhibit distinct technical implementations and risk profiles. Fiat-collateralized stablecoins involve centralized operational management, presenting risks of administrative privilege abuse and access control deficiencies [58]. Crypto-collateralized stablecoins depend heavily on oracles and liquidation mechanisms, making oracle manipulation their primary attack vector [38]. Algorithmic stablecoins rely on supply-demand adjustments, facing severe risks of market manipulation and mechanism failures [61]. The catastrophic collapse of Terra Luna in 2022 demonstrated how algorithmic design flaws, exacerbated by market panic, could vaporize over $40 billion in market value and trigger industry-wide systemic contagion.

Under the current threat landscape, isolated vulnerability detection or static contract auditing is inadequate against complex attack scenarios [49]. As composite attacks leveraging flash loans and oracle manipulation grow increasingly sophisticated, integrating artificial intelligence for real-time anomaly detection and dynamic monitoring has become an imperative trajectory for enhancing system security [42, 47, 48]. Traditional pre-deployment static audits are fundamentally blind to novel, post-deployment composite attacks [30]. Conversely, a dynamic defense model can identify anomalous behaviors in real-time during an ongoing attack sequence and execute immediate interceptions, effectively bridging the inherent gaps of static auditing [65].

Therefore, a systematic investigation into the security vulnerabilities, attack patterns, and dynamic defense mechanisms of stablecoin smart contracts not only holds substantial theoretical value but also carries critical practical significance for safeguarding the stability of the DeFi ecosystem and mitigating systemic financial risks.

The main contributions of this paper are as follows:

- **Systematic Threat Modeling for Stablecoin Ecosystems:** We systematically dissect the distinct technical mechanisms and risk profiles of fiat-collateralized, crypto-collateralized, and algorithmic stablecoins.
- **Full-Lifecycle Dynamic Defense Architecture:** Moving beyond traditional static auditing, we propose a multi-layered, dynamic defense architecture that spans the pre-deployment, mid-execution, and post-incident phases.
- **Real-Time Anomaly Detection via Mempool Monitoring:** We introduce a novel node-level defense mechanism deploying a Bidirectional Long Short-Term Memory network directly at the mempool layer. Our model effectively extracts multi-dimensional on-chain temporal features to intercept malicious payloads before block inclusion.

2 Background

2.1 Smart Contracts

Smart contracts are self-executing protocols whose execution primarily relies on blockchain virtual machines for state updates and transaction processing [19, 37, 44]. They utilize function calls to alter on-chain states, and the execution process typically involves inputting transactions, executing functions, updating states, and triggering events [9, 59, 60]. This execution paradigm is deterministic and immutable, and all nodes execute the contract code according to identical rules to maintain

consistent outcomes. However, this also results in the long-term exploitability of code vulnerabilities. Once a contract is deployed, any inherent vulnerabilities persist indefinitely, unless the contract supports upgradability, they cannot be patched [4]. Within this execution model, smart contracts can perform external calls, a feature that lays the foundation for inter-contract interactions[22]. This allows disparate contracts to invoke one another, thereby facilitating the development of complex financial applications. Nevertheless, this mechanism concurrently introduces security risks. The primary issue is that upon invoking an external contract, execution sequence errors may occur during state updates, a scenario that constitutes the fundamental cause of reentrancy attacks[34].

Taking the classic "The DAO" attack as an example, attackers constructed a malicious contract that triggered a fallback function during the target contract's transfer operation [45, 67]. The original function was repeatedly invoked before the state was updated, allowing the attackers to extract funds multiple times [24, 63]. This case illustrates a critical security flaw in the smart contract execution model that a sequential dependency exists between state updates and external calls. In the Ethereum Virtual Machine (EVM) execution model, an external call suspends the execution of the current function and redirects control to the external contract's code [55]. Only after the external call returns does the current function resume execution. Consequently, this affords attackers the opportunity to insert malicious logic and repeatedly invoke the current function before its state update is finalized.

Security risks also emerge when smart contracts perform numerical computations. Early versions of Solidity lacked automatic overflow checks, meaning integer overflows or underflows could result in erroneous balance calculations [7, 66]. For instance, during the minting or burning of tokens, if numerical ranges are not strictly bounded, attackers can construct extreme inputs to subvert system constraints. Specifically, when a larger number is subtracted from a smaller one, an integer underflow occurs, yielding a substantially large positive value [1]. This enables attackers to bypass balance verification mechanisms and execute illicit fund withdrawals. Although the compiler has incorporated default overflow checks since Solidity version 0.8.0 [51], such vulnerabilities remain prevalent in legacy contracts or in instances where developers have manually disabled these checks.

Access control mechanisms constitute another major security domain. Smart contracts typically employ modifiers to restrict permissions for sensitive operations, such as minting, burning, and contract upgrading, to ensure that only authorized administrators can invoke them [5, 32, 64]. However, if the permission architecture is flawed or lacks rigorous validation, attackers can invoke critical functions to seize control of the entire system [33]. Notably, the initialization functions of many contracts lack adequate permission checks [23]. In such scenarios, attackers can preemptively execute the initialization function post-deployment, designate themselves as administrators, and subsequently take over the contract.

2.2 Stablecoin

Based on their underlying value-backing mechanisms, stablecoins are primarily classified into three categories, including fiat-collateralized, crypto-collateralized, and algorithmic stablecoins, as in Figure 3. These distinct types exhibit significant variations regarding smart contract design complexity and security risks, consequently presenting entirely different attack surfaces.

2.2.1 ***Fiat-Collateralized Stablecoins.*** Fiat-collateralized stablecoins rely on centralized institutions to custody fiat currency and issue an equivalent volume of tokens on the blockchain. Users deposit fiat currency into the issuing institution's bank account, and the issuer subsequently mints an equivalent amount of stablecoins on-chain for the user [29]. During a redemption operation, the issuer burns the on-chain stablecoins and transfers the fiat currency back to the user's bank account, thereby maintaining the price peg [28]. The smart contract functionality for this category

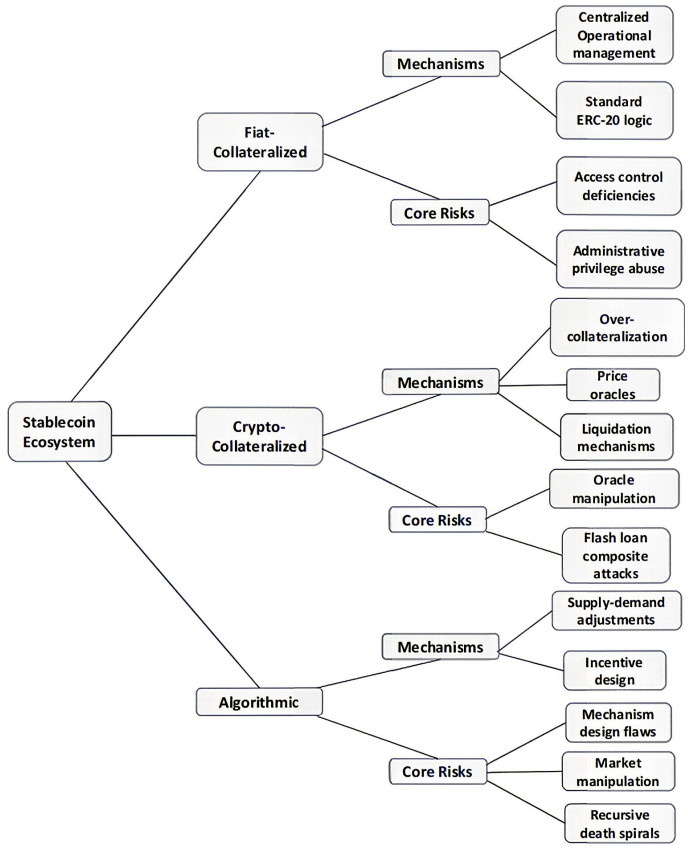

Fig. 1. Taxonomy and Risk Profiles of the Stablecoin Ecosystem.

of stablecoins is relatively straightforward, primarily executing token minting, burning, and balance management. The core architecture typically adheres to the standard ERC-20 logic, augmented with basic access control mechanisms [43]. However, the system's security risks are heavily concentrated in these access controls. Because the permissions for minting and burning are exclusively governed by a centralized issuer, any anomaly in permission management can result in severe security compromises.

In several practical instances, flawed permission management designs have allowed attackers or malicious internal accounts to invoke critical administrative functions, resulting in illicit token issuance or parameter tampering [50]. Such vulnerabilities typically stem from deficient permission validation logic or ambiguous role-based access control (RBAC). For example, if the minting function fails to impose rigorous identity restrictions on the caller, attackers can effortlessly generate massive quantities of stablecoins, severely destabilizing the system [21]. Furthermore, upgradable contract mechanisms introduce supplementary risks. In the event of an access control failure, attackers can exploit logic contract substitutions to assume total governance over the system.

Consequently, the principal risks associated with fiat-collateralized stablecoins can be summarized as access control deficiencies and the abuse of centralized privileges.

2.2.2 ***Crypto-Collateralized Stablecoins.*** Crypto-collateralized stablecoins, epitomized by DAI, operate on the core logic of generating stablecoins by collateralizing mainstream cryptocurrency assets. This model typically necessitates over-collateralization to mitigate the high volatility of

crypto asset prices [58]. For instance, a user might collateralize $150 worth of ETH to generate $100 in stablecoins, with the excess collateral serving as a buffer against ETH price depreciation. The core mechanisms encompass collateral management, loan generation, collateralization ratio computation, and liquidation protocols. When the price of the collateralized asset drops—causing the collateralization ratio to fall below a predefined safety threshold—the system automatically liquidates the user's collateral to repay the debt, thereby ensuring systemic solvency.

In this paradigm, smart contracts manage complex state transitions and computations, relying heavily on price oracles to supply real-time asset price data. Because the system requires instantaneous collateral prices to calculate ratios and determine liquidation triggers, this external dependency constitutes a primary attack vector. Since oracle price data is sourced from off-chain entities or cross-chain decentralized exchanges (DEXs), attackers can manipulate these data feeds to subvert the system's evaluative logic [41]. Several paradigmatic attack cases demonstrate that malicious actors can manipulate asset prices on DEXs, distort oracle data, and ultimately alter collateralization ratio computations. In certain exploits, attackers execute large-volume trades to artificially inflate an asset's price within a narrow time window. This deceives the system into miscalculating the asset's collateral value as artificially high, enabling the attacker to borrow stablecoins far exceeding legitimate collateral constraints before swiftly exiting the protocol.

Attackers frequently leverage flash loans to close the loop on these attack vectors [16]. First, they utilize a flash loan to acquire massive liquidity, allowing them to borrow hundreds of millions of dollars in a single transaction without upfront collateral. Second, they manipulate prices in low-liquidity markets by deploying these borrowed funds to rapidly purchase assets and artificially drive up prices. Third, this price manipulation cascades into the oracle data, causing the system to perceive the collateral's price as extraordinarily high. Finally, this triggers the borrowing or liquidation logic, allowing the attacker to borrow a massive volume of stablecoins against the overvalued collateral and repay the flash loan within the same transaction, thereby realizing a substantial arbitrage profit.

2.2.3 ***Algorithmic Stablecoins.*** Algorithmic stablecoins operate without relying on collateralized assets. Instead, they regulate token supply via smart contracts to achieve price stability. Their core mechanisms include price feedback systems, inflation and deflation adjustments, and incentive design. When the stablecoin's price exceeds its peg, the system mints additional tokens, diluting the supply to drive the price down. Conversely, when the price falls below the peg, the system burns tokens, restricting supply to drive the price up. The system theoretically maintains price stability through this algorithmic supply-and-demand dynamic.

However, these stablecoins face substantial operational risks. In a typical failure scenario, a decline in market confidence triggers user sell-offs, causing the price to drop. The system attempts to maintain the peg by minting more tokens, but this hyperinflationary issuance further dilutes value, exacerbating the price decline and intensifying panic selling. This creates a recursive downward spiral that ultimately leads to systemic collapse. A highly representative incident is the 2022 Terra (LUNA) crash, wherein the price of UST plummeted from its peg to less than $0.01 within a matter of days. This resulted in a total systemic collapse, inflicting losses exceeding $40 billion. Analyzed from a technical perspective, such failures are not merely the result of codebase vulnerabilities, but rather the consequence of the tight coupling between economic model design and contract execution logic. The primary catalyst for the Terra collapse was the inherent structural deficiency of the uncollateralized algorithmic stablecoin model, compounded by panic selling triggered by a total collapse in market confidence. Because algorithmic stablecoins rely on highly complex contract logic, vulnerabilities can frequently exist at the code level, which further amplifies the overall risk profile. Therefore, the core risk of algorithmic stablecoins lies in the interaction between

mechanism design flaws and the amplifying effects of market behavior. This category of stablecoins possesses the broadest attack surface, encompassing both code-level vulnerabilities and economic mechanism exploits.

2.3 Smart Contract Security Threat Model

2.3.1 ***Attack Surface Analysis.*** Based on practical cases, the attack surfaces of stablecoin systems can be categorized into the following domains:

- **Code-Level Vulnerabilities**: This includes reentrancy attacks and integer overflows, which are directly exploited due to flaws inherent in the contract's execution logic.
- **Access Control Vulnerabilities**: Instances such as unauthorized access allow attackers to govern critical system functions. These vulnerabilities are predominantly found in fiat-collateralized stablecoins, where attackers can directly hijack core administrative functions.
- **External Dependency Risks**: A prime example is oracle manipulation, where attackers indirectly compromise the entire system by maliciously influencing input data. Such vulnerabilities frequently manifest in crypto-collateralized stablecoins due to the system's heavy reliance on external price feeds.
- **Economic Model Vulnerabilities**: This involves failures in algorithmic stability driven by incentive mechanisms or market behaviors. These vulnerabilities predominantly exist in algorithmic stablecoins, where attackers exploit structural defects within the economic model itself.
- **Composite Attack Vectors**: These involve the synergistic application of multiple techniques, such as the combination of flash loans, oracle manipulation, and liquidation attacks. Attackers chain multiple vulnerabilities together to achieve sophisticated and devastating attack outcomes.

2.3.2 ***Typical Attack Paths.*** Based on empirical cases, typical attack paths within stablecoin systems can be abstracted, as shown in Figure 2. These trajectories recursively appear across numerous attack incidents and exhibit significant commonalities:

- **Reentrancy Attack Path**: The execution sequence typically initiates with an external call, followed by the activation of a fallback function [56]. This enables a recursive invocation of the target function prior to state updates, ultimately culminating in unauthorized fund extraction.
- **Oracle Manipulation Path**: This exploit generally begins with targeted market manipulation that induces a significant asset price deviation [10]. The distorted data is subsequently ingested during an oracle update, leading to a critical system miscalculation.
- **Flash Loan Composite Attack Path**: This trajectory commences with massive fund acquisition via a flash loan without upfront collateral [62]. The acquired liquidity is then deployed for market manipulation to trigger vulnerable contract logic, concluding with a profitable exit and loan repayment within the same atomic transaction.
- **Privilege Abuse Path**: This path involves initial unauthorized privilege acquisition, which facilitates subsequent arbitrary parameter modification, thereby granting the attacker complete control over the system's behavior [17].

2.4 Security Protection Technologies

Addressing the aforementioned attack patterns, existing security protection technologies are primarily distributed across various phases of the development lifecycle. However, empirical attack cases have revealed their inherent limitations, demonstrating an inability to fully counter the sophisticated exploits prevalent in stablecoin scenarios.

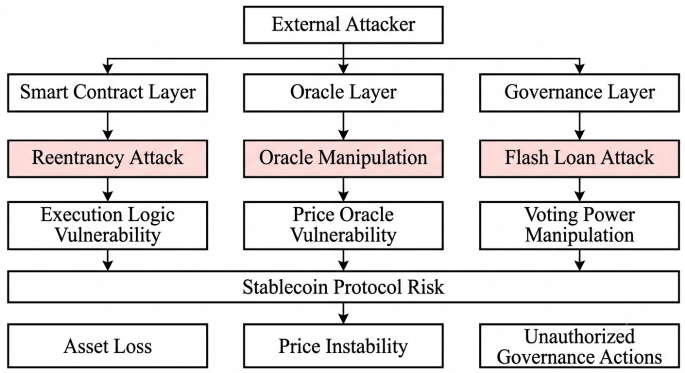

Fig. 2. Threat Model of Security Attacks in Stablecoin Systems.

2.4.1 ***Static Analysis Methods.*** Static analysis is a vulnerability detection method targeting contract source code or bytecode [12]. It detects vulnerable behavioral patterns by analyzing the code's control flow and data flow, thereby uncovering predefined vulnerabilities. The primary advantage of static analysis lies in its ease of automation and its efficacy in identifying vulnerabilities within a given set of contract code. Nevertheless, static analysis possesses distinct limitations. First, it cannot detect vulnerabilities that emerge dynamically during contract execution. For instance, when a contract utilizes external data for computations, malicious data inputs might trigger vulnerabilities that static analysis typically fails to capture. Second, static analysis is often ineffective in evaluating dynamic calls or complex logic within the contract.

2.4.2 ***Fuzzing Methods.*** Fuzzing conducts vulnerability detection by injecting random, invalid, or malicious data into a contract [18]. This method is capable of uncovering edge cases within smart contracts and identifying potential security issues. However, in practical applications, neither dynamic analysis nor fuzzing can achieve complete code path coverage. In complex contracts, certain vulnerabilities are inevitably overlooked, and the results often suffer from false positives and false negatives, thereby complicating the analysis process.

2.4.3 ***Formal Verification Methods.*** Formal verification for smart contract vulnerabilities is a technique rooted in mathematics and logical reasoning, designed to mathematically guarantee the absence of security flaws during contract execution [15]. Smart contracts can employ formal verification across their lifecycle, utilizing this technique throughout both the design and development phases. Code-level formal verification can implement dual verification within a unified environment using both source code and compiled bytecode. The source code undergoes transformation for verification, while the compiled bytecode is decompiled to verify low-level properties. These two verification approaches utilize equivalence proofs to guarantee functional and operational consistency.

Although this method provides robust security guarantees, its modeling and proving processes are highly complex, leading to increased difficulties in implementation and comprehension. Consequently, its application in stablecoin systems is limited. Formal verification can only prove properties at the code level, it cannot mathematically guarantee the security of the economic model, thus rendering it incapable of preventing economic-layer exploits such as oracle manipulation or composite flash loan attacks.

2.4.4 ***Intermediate Representation.*** Intermediate Representation (IR) is a data structure utilized to represent a program's intermediate state or form [6]. It is typically generated after the source code undergoes lexical and syntax analysis phases, and is employed prior to target code generation, remaining independent of specific programming languages and target platforms. Common intermediate representations include Abstract Syntax Trees (AST), Three-Address Code, and Static Single Assignment (SSA) form. Different compilers and toolchains may employ distinct IRs. Within a compiler, the IR plays a crucial role during the optimization and analysis phases. By analyzing and transforming the IR, various optimization measures, such as constant folding, loop optimization, and inlining, can be implemented. Simultaneously, IR can also be utilized to generate target code, converting the optimized program into platform-specific opcodes or bytecode. Smart contracts are generally written in high-level languages. However, due to the complex structure and semantics of the source code, direct vulnerability detection on the source code often encounters significant challenges. Therefore, translating smart contract source code into an intermediate representation can simplify the vulnerability detection process and facilitate the application of static analysis techniques.

2.4.5 ***Machine Learning Methods.*** The objective of machine learning is to enable computers to make predictions and decisions to solve problems based on past experience, through data learning and pattern recognition, without requiring explicit programming instructions. In recent years, machine learning has been increasingly adopted for smart contract vulnerability detection, capable of automatically mining potential vulnerabilities and security issues by learning from vast datasets of smart contracts [4]. During the training process, algorithms automatically discover patterns and regularities within the data, using them as a basis to adjust and update model parameters. Through continuous iterative learning, the model can be progressively optimized and improved, enhancing its generalization capability on unseen data. Machine learning plays a crucial role in various aspects of smart contract vulnerability detection, including feature extraction, anomaly detection, vulnerability classification, and generative training. At the same time, applying machine learning to this task still faces significant challenges. The complexity and highly dynamic nature of smart contracts make vulnerability detection increasingly difficult. Furthermore, the lack of sufficient training data and high-quality labeled data also constrains the efficacy of model training.

3 Defense Methods for Stablecoin Smart Contracts

3.1 Full-Lifecycle Dynamic Defense Architecture

Stablecoin smart contracts face complex security threats, and traditional static defense methods generally fail to keep pace with real-time on-chain attacks [39]. To address this, this paper proposes a defense architecture, as shown in Figure 3 comprising pre-deployment screening, mid-execution interception, and post-incident response. This model leverages the synergy of multiple technologies to integrate security defenses throughout the complete lifecycle of smart contracts.

3.1.1 ***Pre-Deployment Defense Phase.*** Before the contract code is deployed to the mainnet, this phase primarily focuses on eliminating existing defects in code logic and business semantics. This stage utilizes static code scanning to detect common vulnerabilities such as integer overflows and reentrancy risks, and applies formal verification techniques to mathematically prove that the core business logic maintains state consistency. Concurrently, automated and manual security audits conducted by professional third-party organizations establish a fundamental security baseline prior to protocol launch, thereby mitigating potential risks at the underlying code level.

3.1.2 ***Mid-Execution Defense Phase.*** During the deployment and operational phases of the contract, the focus shifts to addressing zero-day vulnerabilities and complex composite attacks. This

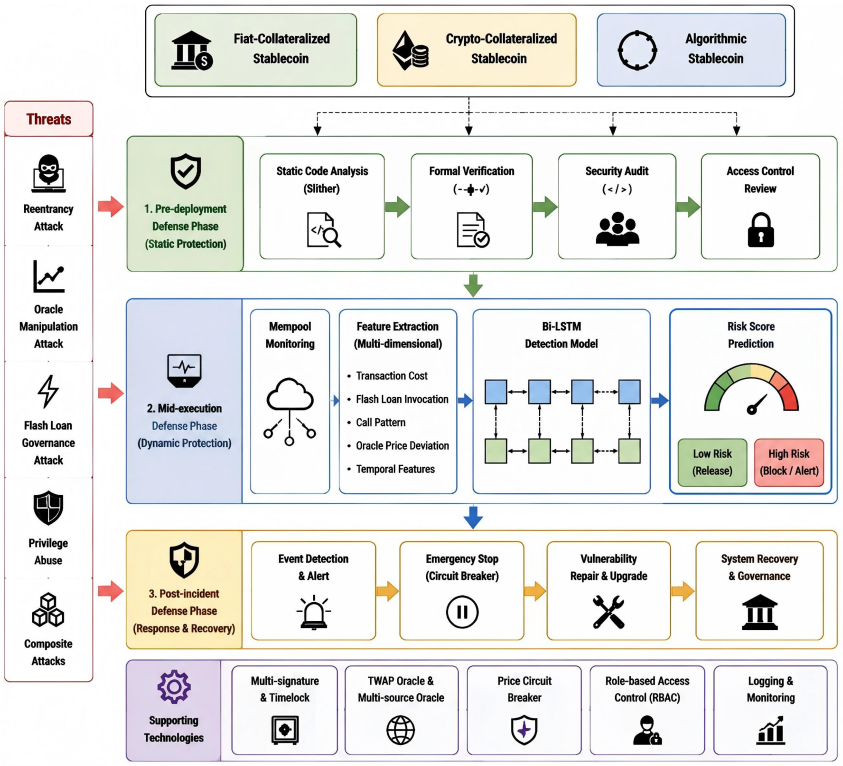

Fig. 3. Overview of the proposed lifecycle-based stablecoin security framework.

phase designates the node memory pool (Mempool) as the central monitoring layer and employs deep learning methods, such as Bi-LSTM, to construct an anomaly detection engine. This engine performs millisecond-level temporal feature extraction and risk inference on transaction sequences that have not yet been packed into the blockchain. Upon detecting attack payloads with a high probability of risk, the system automatically triggers an RPC-level hard interception or issues a suspend command to the target contract. This prevents the attack from causing substantial state transitions and successfully blocks the threat.

3.1.3 ***Post-Incident Defense Phase.*** In the event of a security breach under extreme circumstances, the priority is to contain the scope of losses and facilitate business recovery. This phase primarily relies on global pause (circuit breaker) functionalities and emergency proxy upgrade mechanisms predefined during the initial contract design. Utilizing an event-driven on-chain state monitoring model, if an unexpected massive outflow of funds occurs, the system automatically activates contingency plans to isolate the compromised liquidity pool. Subsequently, by employing automated vulnerability repair assistive technologies combined with manual intervention, the vulnerabilities can be rapidly patched, restoring the system to normal operational status.

3.2 Differential Defense Strategies

Given the distinct risk profiles and attack surfaces inherent in various stablecoin architectures, this paper proposes differential defense strategies tailored to the three primary stablecoin paradigms, systematically mitigating their respective core vulnerabilities.

3.2.1 ***Privilege Isolation and Auditing.*** The primary risks associated with fiat-collateralized stablecoins are access control deficiencies and privilege abuse; thus, defense efforts must center on rigorous permission management.

- **Granular RBAC**: Decouple permissions for minting, burning, and upgrading into distinct roles to eliminate the existence of a super-administrator, ensuring each role adheres to the principle of least privilege.
- **Multi-Signature and Time-Lock Mechanisms**: Mandate multi-signature approvals for all sensitive operations and integrate time locks to enforce a 24 to 48 hour execution delay. This provides users with a sufficient withdrawal window, effectively deterring the immediate execution of malicious or compromised administrative commands.
- **Periodic Permission Audits**: Conduct routine inspections of contract permission configurations to verify the absence of unauthorized functions, thereby preventing privilege escalation vulnerabilities.
- **Transparent Reserve Assets**: Require the regular cryptographic disclosure of proof of reserves to prevent the misappropriation of collateralized funds by the issuing entity.

3.2.2 ***Oracle Security and Liquidation Protection.*** The core vulnerability of crypto-collateralized stablecoins lies in oracle manipulation. Thus, defense mechanisms must prioritize oracle data integrity.

- **Mandatory TWAP (Time-Weighted Average Price)**: Enforce the use of TWAP and strictly prohibit reliance on spot prices, fundamentally mitigating the system's susceptibility to flash price manipulation.
- **Multi-Source Oracle Aggregation**: Deploy mature decentralized oracles to aggregate price feeds from multiple independent data sources, thereby eliminating the single-point-of-failure risks associated with centralized data feeds.
- **Price Circuit Breakers**: Automatically suspend liquidation and borrowing functionalities when asset price volatility exceeds a predefined threshold, defending the protocol against extreme price manipulation events.
- **Liquidation Safeguards**: Impose strict upper bounds on the maximum volume of a single liquidation event to prevent attackers from exploiting liquidation mechanics for predatory arbitrage.

3.2.3 ***Economic Model Stress Testing and Mechanism Defenses.*** The critical risks characterizing algorithmic stablecoins are inherent economic design flaws and the propensity for recursive death spirals. Therefore, protective strategies must focus on the resilience of the economic model.

- **Economic Model Stress Testing**: Prior to deployment, subject the economic model to rigorous computational stress testing under extreme scenarios to simulate black swan market events. This evaluates model stability and ensures resilience against death spirals during severe market distress.
- **Algorithmic Circuit Breakers**: Automatically halt token supply adjustments when the stablecoin's price deviates from its peg beyond a critical threshold, thereby arresting the systemic feedback loop that exacerbates a death spiral.
- **Code-Level Security Auditing**: Given the high structural complexity of algorithmic stablecoin logic, mandate exhaustive and rigorous code audits to eradicate underlying vulnerabilities such as reentrancy and integer overflows.
- **Incentive Mechanism Optimization**: Refine the incentive structures governing algorithmic supply-and-demand regulation to preclude attackers from exploiting these mechanisms for malicious, risk-free arbitrage.

3.3 AI-Based Real-Time Anomaly Detection System

Flash loan composite attacks and oracle manipulation events, which frequently occur within the smart contract ecosystem, exhibit formidable destructive potential. Under the operational paradigms of mainstream blockchains such as Ethereum, conventional defense mechanisms primarily rely on retrospective smart contract security audits or blacklist-based interception using the historical invocation records of known malicious addresses. However, contemporary composite attacks are typically executed within an extremely narrow time window, seamlessly completing the entire lifecycle, from initial fundraising and multi-path cross-contract invocations to the manipulation of underlying liquidity pool prices and the final risk-free profitable exit. This instantaneous nature renders transition responses largely ineffective.

In this paper, we design a preemptive AI anomaly detection system deployed at the Mempool level of underlying blockchain nodes. To process pending transactions prior to the execution of state transitions by the Ethereum Virtual Machine (EVM), this system utilizes a deep neural network to extract temporal features from invocation sequences, as shown in Figure 4. It performs millisecond-level preemptive inference to facilitate precise, physical-level blocking of malicious transactions before they are recorded on the chain.

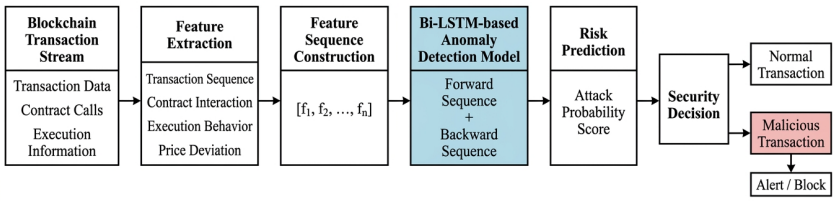

Fig. 4. AI-based Real-time Transaction Detection Workflow for Stablecoin Security.

3.3.1 ***Multidimensional Reconstruction of On-Chain Interaction Feature Engineering.*** Malicious on-chain transactions often manifest as fund fragmentation and redundant nested transfers, utilizing these patterns to evade static rule-based detection methods. In this paper, we construct a transaction feature vector space encompassing five dimensions to capture the primary characteristics of attack behaviors. Utilizing statistical principles, this study generates a simulated on-chain dataset comprising 100,000 samples for model training and validation, with a positive-to-negative sample ratio set at 95% normal transactions to 5% malicious attacks. These five-dimensional features are categorized into three major classes based on business logic. The specific extraction contents and selection criteria are detailed as follows:

- **Basic Transaction and Execution Cost Features**: The extracted features include the transaction principal size (Amount), the maximum gas fee consumed (Gas Limit), and the encoded length of the input data payload (Input Data Length). The rationale for selecting these features is that flash loan composite attacks necessitate cross-contract invocations across multiple DeFi protocols, which typically involve deep function nesting, external state calls, and multiple modifications via the underlying storage instruction (SSTORE). Compared to routine on-chain transfers or single DEX token swaps, attack transactions consume significantly more gas and feature longer input code payloads. To secure substantial arbitrage profits, the principal volume transferred is generally massive. Attackers may attempt to obfuscate these three metrics to masquerade as benign transactions. In the experiments, these three features were modeled using a normal distribution with a large variance, a mean of 0.5, and a standard deviation of 0.3. This distribution ensures an overlap between normal and attack

samples across basic metrics, dictating that the model cannot rely solely on a single numerical threshold for judgment, and must integrate temporal context for accurate classification.

- **Liquidity Flash Loan Invocation Features**: This extracted feature determines whether a flash loan interface is triggered within the transaction and its internal call stack. The selection is based on the premise that oracle manipulation relies heavily on massive, uncollateralized capital to disrupt market price equilibrium. Flash loans grant attackers access to substantial liquidity at a minimal cost, serving as the primary funding source for most serial composite attacks. Thus, this paper identifies it as a critical dimension. In reality, legitimate high-frequency quantitative arbitrage bots also legally invoke flash loans to eliminate price discrepancies across DEXs. Therefore, this feature is modeled using a binomial distribution probability. It is assumed that attackers, requiring high leverage, have an 85% probability of utilizing flash loans, whereas normal arbitrageurs possess an 8% probability of legitimate invocation. This probability assignment objectively reflects the presence of confounding factors in real-world business logic and prevents the model from falling into the overfitting fallacy of equating any flash loan usage with an attack.
- **Oracle Price Deviation Features**: The extracted features encompass the anticipatory calculation of the underlying liquidity pool's AMM (Automated Market Maker) token swap price slippage that the current transaction execution might induce. This selection is grounded in the constant product formula of AMMs, where a massive unilateral selling pressure within a short timeframe distorts the exchange ratio. A significant price deviation is the primary consequence of an attacker executing a "buy low, sell high" strategy to exploit oracle price discrepancies, serving as a vital quantitative indicator of malicious manipulation. Normal transactions generate relatively minor price slippage. The experiment simulates this using an exponential distribution with a scale parameter set to 0.15, indicating that while most arbitrage slippages are minute, extreme deviations occur under a long-tail effect. To guarantee an arbitrage margin, malicious attacks induce pronounced price anomalies. This paper analyzes this using a normal distribution model. Extracting this feature quantifies the extent of damage the transaction inflicts on market prices.

3.3.2 ***Cost-Sensitive Detection Model Based on Bi-LSTM.*** Addressing the aforementioned feature sequences characterized by temporal dependencies and distributional noise, traditional static rules or shallow machine learning algorithms struggle to effectively capture contextual relationships across the temporal dimension. In this paper, we adopt the Bidirectional Long Short-Term Memory (Bi-LSTM) network as the core architecture of the anomaly detection model.

Bi-LSTM comprises both forward and backward layers for information propagation. Within a complex composite attack, a complete transaction typically entails early-stage borrowing for fundraising, mid-stage price manipulation, and late-stage account reconciliation and withdrawal. The Bi-LSTM can concurrently extract the bidirectional temporal correlations of these sequential steps, thereby more accurately capturing the complete transactional semantics. The system configures the input dimension and establishes a 2-layer bidirectional hidden network, setting the hidden node dimension to 16. The network extracts the output of the sequence's final time step, processing it through a fully connected layer and a Sigmoid activation function to output a continuous numerical value ranging from 0 to 1, representing the probability of anomalous risk for the transaction.

On-chain data inherently exhibits a state of class imbalance between positive and negative samples. As previously established, normal transactions account for 95% of the total dataset. If a conventional cross-entropy loss function is employed directly for network training, the model is prone to converging into a local optimum. Specifically, leaning towards predicting the vast majority

of samples as normal to achieve a high overall Accuracy. This numerical bias renders the model highly inefficient at identifying sparse attack samples, causing the recall rate to approach zero, thus stripping the model of its practical defensive utility.

To address this imbalance at the algorithmic level, this paper introduces a Cost-Sensitive Learning paradigm. During the computation of backpropagation gradients, the system utilizes a binary cross-entropy loss function incorporating a positive sample penalty weight, calibrated according to the a priori ratio of normal to malicious samples. Within the defined experimental distribution, malicious samples are assigned a misclassification penalty weight approximately 19 times higher than that of normal samples. The learning rate of the Adam optimizer is set to 0.005, increasing the cost of misclassifying minority class samples during the training process. By adopting this strategy, this paper prioritizes enhancing the recall rate for anomalous transactions, even at the expense of a marginal reduction in precision, thereby minimizing the risk of false negatives in practical deployments.

3.3.3 ***Automated Interception Mechanism Based on Mempool Monitoring.*** During the inference phase, the neural network model predominantly performs matrix operations characterized by low computational latency, establishing the feasibility of deploying it on Ethereum Remote Procedure Call (RPC) nodes or validator nodes.

This paper deploys the anomaly detection engine at the ingress of the node's transaction queue (Tx-Queue). A real-time mempool monitoring daemon, monitor_mempool, is designed to validate its operational efficacy. This daemon continuously simulates the influx of on-chain transaction data, dynamically injecting feature fluctuations that align with the previously established statistical distributions during runtime. To account for underlying inference latency, the system introduces a random temporal jitter ranging from 0.15 to 0.22 seconds, simulating data transmission and deserialization delays inherent in real-world distributed networks.

Once the node captures and extracts the transaction features in real time, the data is fed into the Bi-LSTM network loaded with pre-trained weights. Operating with gradient computation disabled (torch.no_grad()), the model performs forward propagation and outputs the systemic risk confidence score for the transaction. To mitigate the false positive rate associated with legitimate high-frequency arbitrage operations, the system sets the risk interception threshold at 85%. Based on this threshold, the system defines the following two automated processing protocols:

- **Normal Release**: When the attack confidence is below 85.00%, the system determines that the current transaction conforms to standard on-chain interaction patterns. This spectrum includes ordinary address transfers and conventional quantitative arbitrage trades that may induce slight price slippage. In this state, the system merely records a release log in the background and broadcasts the transaction data normally to the peer-to-peer (P2P) network or forwards it to the miner node pool to await consensus packing, without interfering with its standard execution flow.
- **High-Risk Alert and Automated Interception Closed-Loop**: When the attack confidence reaches or exceeds 85.00%, the model detects anomalies such as irregular gas consumption, flash loan invocations, and oracle price deviations during its comprehensive analysis. Within a sub-second window, the system initiates an automated closed-loop response consisting of Local Mempool Filtering and On-Chain Automated Defense.

4 Experiment Evaluation

In this section, we evaluate the effectiveness of the proposed stablecoin security framework through two groups of experiments. The first experiment focuses on validating the effectiveness of the proposed smart contract defense mechanisms against representative stablecoin-related attacks.

The second experiment evaluates the performance of the proposed AI-based real-time anomaly detection system in identifying malicious transaction behaviors. Specifically, we aim to answer the following research questions:

RQ1: Can the proposed defense mechanisms effectively mitigate representative attacks targeting stablecoin smart contracts?

RQ2: Can the proposed Bi-LSTM-based anomaly detection model accurately identify malicious transactions while maintaining low inference latency?

To answer these questions, we construct a local Ethereum-compatible experimental environment and conduct attack reproduction experiments, defense verification experiments, and AI model performance evaluations.

4.1 Experimental Setup

All experiments are conducted on a local Ethereum-compatible testing environment. The smart contract experiments are implemented using Solidity and executed on the Foundry framework. The AI-based anomaly detection model is developed using PyTorch. The detailed hardware and software configurations are summarized in Table 1.

Table 1. Experimental Environment Configuration

Component	Configuration
Operating System	Ubuntu 22.04
Blockchain Framework	Foundry v1.5.1
Smart Contract Language	Solidity
AI Framework	PyTorch
Python Version	3.12.3
CPU	16-core Processor
Memory	32 GB RAM

All smart contract experiments are conducted in a local Ethereum-compatible testing environment based on Foundry. Foundry is selected because it provides efficient smart contract compilation, deployment, and testing capabilities, which facilitates the reproduction of complex DeFi attack scenarios. The smart contracts used in the experiments are implemented using Solidity. Three representative attack scenarios are constructed, including reentrancy attacks, oracle manipulation attacks, and flash loan-based governance attacks, as shown in Figure 5. For each attack scenario, we first deploy a vulnerable contract implementation and reproduce the corresponding exploit process. Then, the proposed defense mechanism is integrated into the contract and evaluated under the same attack conditions.

For the anomaly detection experiment, the proposed Bi-LSTM model is implemented using PyTorch. Since publicly available datasets containing sufficient labeled stablecoin attack transactions are limited, we construct a simulated transaction dataset based on realistic DeFi attack characteristics. The dataset contains 100,000 transaction samples, including 95,000 normal transactions and 5,000 malicious transactions. Each transaction is represented using multi-dimensional behavioral features, including transaction execution cost, flash loan invocation patterns, oracle price deviation, and contract interaction behaviors.

The dataset is randomly divided into training and testing sets with a ratio of 80% and 20%, respectively. The training set is used for model optimization, while the testing set is used for evaluating the generalization capability of the detection model.

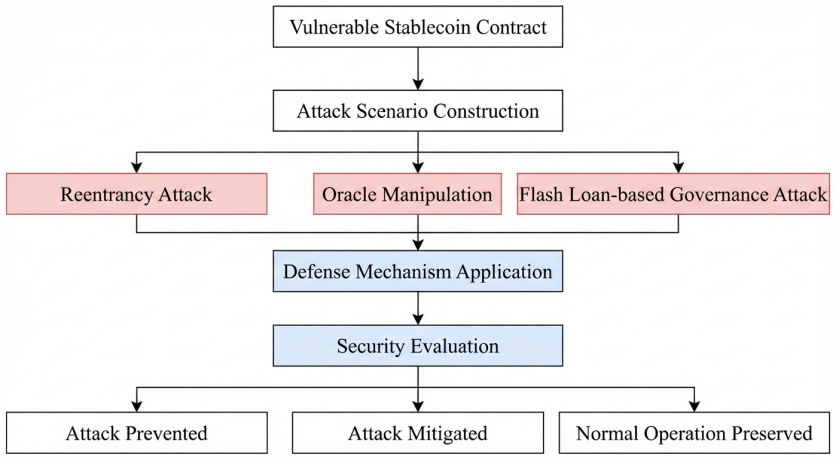

Fig. 5. Experimental Workflow for Stablecoin Attack Defense Evaluation.

4.2 Evaluation of Smart Contract Defense Mechanisms

To answer RQ1, we evaluate the effectiveness of the proposed smart contract defense mechanisms against representative attacks targeting stablecoin protocols. Although stablecoin systems differ in their underlying mechanisms, existing attacks generally exploit three fundamental weaknesses, including insecure contract execution logic, unreliable external data sources, and vulnerable economic governance mechanisms. Therefore, we select three representative attack scenarios for evaluation, as shown in Table 2, including reentrancy attacks, oracle manipulation attacks, and flash loan-based governance attacks.

For each attack scenario, we first deploy a vulnerable smart contract implementation and reproduce the corresponding attack process in the local Ethereum-compatible environment. Subsequently, the proposed defense mechanism is integrated into the vulnerable contract, and the same attack procedure is executed again to evaluate whether the attack can be effectively mitigated. The effectiveness of the defense mechanism is evaluated based on two criteria, including whether the attacker can successfully complete the malicious transaction and obtain unauthorized benefits, and whether the proposed mechanism can prevent abnormal state transitions while preserving normal contract functionality.

4.2.1 Reentrancy Attack Evaluation. Reentrancy attacks represent one of the most common vulnerabilities in Ethereum smart contracts, where an attacker repeatedly invokes a vulnerable external function before the contract updates its internal state. In stablecoin systems, such vulnerabilities may result in unauthorized withdrawal of collateral assets or liquidity drainage from protocol pools.

Attack Setup. To evaluate the effectiveness of the proposed defense mechanism, we construct a vulnerable stablecoin liquidity pool contract containing a withdrawal function with an insecure execution order. Specifically, the contract transfers assets to the external caller before updating the user's internal balance, creating a reentrancy opportunity.

An attacker contract is deployed to exploit this vulnerability through a fallback function. Once the attacker initiates the withdrawal request, the fallback function is repeatedly triggered, allowing multiple withdrawals within a single transaction before the balance state is updated. The liquidity pool is initialized with sufficient assets to simulate a realistic stablecoin reserve environment.

Table 2. Attack Defense Evaluation

Attack Type	Defense Mechanism	Result
Reentrancy Attack	Reentrancy Guard + CEI Principle	Prevented
Oracle Manipulation	TWAP + Multi-source Oracle	Mitigated
Flash Loan Governance	Time-lock Mechanism	Prevented

Defense Mechanism. To mitigate this vulnerability, we integrate a reentrancy protection mechanism based on execution-state locking. Before executing sensitive external calls, the contract maintains a mutex status variable to prevent recursive invocation of protected functions. Furthermore, the withdrawal logic is modified following the checks-effects-interactions principle. The user's balance state is updated before transferring assets to external addresses, ensuring that subsequent recursive calls cannot reuse the original balance state.

Evaluation Result. Without the proposed defense mechanism, the attacker contract successfully performs recursive calls and withdraws funds beyond the authorized balance, demonstrating the feasibility of the reentrancy exploit. After applying the reentrancy protection mechanism, the same attack transaction is rejected during execution because the recursive invocation attempt violates the contract state-lock condition. Meanwhile, legitimate withdrawal operations remain unaffected.

The experimental results demonstrate that the proposed defense mechanism effectively prevents reentrancy asset drainage attacks by restricting abnormal recursive execution paths.

4.2.2 Oracle Manipulation Attack Evaluation. Oracle manipulation attacks are critical threats to decentralized stablecoin systems because incorrect price information can directly affect collateral valuation, liquidation decisions, and token stability mechanisms. Unlike traditional smart contract vulnerabilities, oracle attacks exploit the dependency between on-chain protocols and external market data sources.

Attack Setup. To evaluate the effectiveness of the proposed oracle security mechanism, we construct an oracle manipulation scenario based on an AMM liquidity pool. In the vulnerable implementation, the stablecoin protocol directly relies on instantaneous spot prices obtained from a decentralized exchange. An attacker first acquires a large amount of temporary liquidity and performs a series of token swaps within a short period. These operations introduce significant price deviation in the liquidity pool, causing the oracle to return an abnormal asset price. The manipulated price is subsequently utilized by the stablecoin protocol for collateral valuation and liquidation decisions. As a result, attackers can exploit the incorrect price information to obtain unfair liquidation benefits or manipulate protocol states.

Defense Mechanism. To mitigate oracle manipulation risks, we introduce a TWAP-based oracle mechanism combined with multi-source price aggregation. Instead of directly using instantaneous market prices, the TWAP mechanism calculates asset prices based on historical observations over a predefined time window. Therefore, short-term price fluctuations caused by large transactions cannot immediately influence the reported oracle value. In addition, the multi-source oracle aggregation mechanism collects price information from multiple independent data providers and reduces the impact of a single manipulated price source. A price circuit breaker is further introduced to temporarily suspend sensitive operations when abnormal price deviations exceed predefined thresholds.

Evaluation Result. In the vulnerable oracle implementation, the attacker can significantly modify the AMM pool price within a single transaction sequence, causing the stablecoin protocol to receive inaccurate price information and execute incorrect financial operations. After integrating the proposed oracle protection mechanism, short-term price manipulation fails to immediately affect

the oracle output because the TWAP calculation smooths transient price fluctuations. Meanwhile, abnormal price deviations are detected by the circuit breaker mechanism, preventing potentially harmful liquidation or borrowing operations.

The experimental results demonstrate that the proposed oracle protection strategy effectively improves the resilience of stablecoin systems against short-term price manipulation attacks.

4.2.3 Flash Loan Governance Attack Evaluation. Flash loan-based governance attacks have become an emerging threat in decentralized finance systems. Unlike traditional attacks that exploit programming errors, flash loan attacks exploit the atomic execution characteristics of blockchain transactions, allowing attackers to temporarily acquire large amounts of governance power without long-term capital commitment. In stablecoin protocols, such attacks may allow malicious actors to manipulate governance decisions, modify critical protocol parameters, or execute unauthorized administrative operations.

Attack Setup. To evaluate the effectiveness of the proposed governance protection mechanism, we construct a vulnerable decentralized governance contract combined with a flash loan provider. In the vulnerable implementation, voting power is determined directly based on the current token balance within the same transaction. An attacker can therefore borrow a large amount of governance tokens through a flash loan, temporarily increase their voting weight, submit a malicious proposal, and execute the proposal before the flash loan is repaid. The complete attack process consists of four stages, which include obtaining temporary liquidity through a flash loan, acquiring excessive governance voting power, submitting and approving a malicious proposal, and executing unauthorized governance operations within the same transaction.

Defense Mechanism. To mitigate flash loan governance attacks, we introduce a time-lock-based governance execution mechanism. Unlike the vulnerable design that allows immediate execution after proposal approval, the proposed mechanism separates proposal approval and execution into two independent phases. After a proposal receives sufficient votes, the execution is delayed for a predefined time interval. During this delay period, the protocol can perform additional security verification, and temporary voting power obtained through flash loans becomes invalid because the borrowed assets must be returned before transaction completion. Therefore, attackers cannot maintain sufficient governance influence during the actual execution phase.

4.3 Evaluation of AI-based Real-time Attack Detection

To answer RQ2, we evaluate the effectiveness of the proposed Bi-LSTM-based anomaly detection model in identifying malicious stablecoin transactions. Different from traditional smart contract vulnerability detection methods that analyze source code after deployment, our approach focuses on transaction-level behavioral analysis and aims to detect abnormal activities before transaction confirmation.

4.3.1 Dataset Construction and Experimental Configuration. Due to the limited availability of publicly labeled stablecoin attack transactions, we construct a simulated transaction dataset based on representative DeFi attack characteristics and normal transaction behaviors.

The dataset contains 100,000 transaction samples, including 95,000 normal transactions and 5,000 malicious transactions. The malicious samples cover several representative attack patterns, including abnormal contract interactions, flash loan behaviors, oracle price deviations, and irregular transaction execution patterns. Each transaction is represented as a sequential feature vector containing multiple behavioral attributes extracted from transaction execution information. These features include transaction cost, contract invocation patterns, liquidity borrowing behaviors, price deviation information, and temporal interaction characteristics. The dataset is divided into training

and testing subsets with an 80:20 ratio for 10 rounds of experiments. During model training, a cost-sensitive binary cross-entropy loss function is adopted to alleviate the impact of class imbalance and improve the detection capability for minority attack samples.

4.3.2 Detection Performance Evaluation. We evaluate the detection performance of the proposed model using Accuracy, Precision, Recall, and F1-score. Since attack transactions usually represent only a small proportion of blockchain activities, Recall is considered the most important metric because missing malicious transactions may result in significant financial losses. The experimental results are summarized in Table 3.

Table 3. Performance Evaluation of the Anomaly Detection Model

Experiment No.	Test Set Size (Samples)	Actual Attacks (Samples)	Accuracy (%)	Recall (%)	Precision (%)
1	20000	1004	96.27	97.31	57.57
2	20000	991	97.23	97.48	64.66
3	20000	1015	97.91	95.76	72.11
4	20000	1023	97.02	97.85	63.52
5	20000	1018	95.17	98.53	51.36
6	20000	989	96.56	98.28	59.20
7	20000	1058	96.70	97.83	61.90
8	20000	972	96.74	97.53	60.15
9	20000	1023	95.79	98.44	54.94
10	20000	1020	96.72	97.94	61.14
Average	20000	1011.3	96.61	97.70	60.66

The proposed model achieves an accuracy of 96.61% and a recall of 97.70%, demonstrating its effectiveness in identifying malicious transaction behaviors. The high recall indicates that the model can successfully capture most attack transactions, which is particularly important for security-critical blockchain environments where false negatives may lead to irreversible financial losses. However, the precision value is relatively lower compared with recall. This is mainly caused by the highly imbalanced transaction distribution, where certain legitimate high-frequency DeFi operations may exhibit behaviors similar to malicious activities. This trade-off is acceptable because the proposed system prioritizes minimizing missed attacks in real-time protection scenarios.

4.3.3 Real-time Detection Capability Analysis. Besides detection accuracy, response latency is another critical factor for blockchain security systems. To evaluate the feasibility of real-time deployment, we measure the inference latency of the trained Bi-LSTM model during transaction-level prediction. The average inference latency ranges from 1.5 ms to 2.8 ms for a single transaction feature sequence. The latency measurement includes feature input processing and model forward inference but excludes network transmission overhead. Compared with the block confirmation interval of Ethereum, which usually requires several seconds, the inference time of the proposed model is sufficiently low to support transaction-level risk monitoring before confirmation. These results demonstrate that the proposed anomaly detection mechanism can be integrated into blockchain nodes or RPC gateways to provide proactive transaction screening.

5 Discussion

The experimental results demonstrate that the proposed framework provides an effective and comprehensive security solution for stablecoin smart contracts by combining preventive protection mechanisms with real-time transaction-level anomaly detection. Unlike traditional security approaches that mainly focus on post-deployment vulnerability analysis or isolated attack mitigation, the framework adopts a lifecycle-based defense strategy. The experimental evaluation of reentrancy attacks, oracle manipulation attacks, and flash loan governance attacks shows that different attack surfaces require corresponding protection mechanisms. Specifically, execution-level vulnerabilities can be mitigated through secure coding patterns and runtime protection, while economic and data-related attacks require protocol-level mechanisms such as oracle aggregation and governance delay.

Furthermore, the AI-based detection module complements traditional smart contract security mechanisms by providing proactive protection against transaction-level threats. The Bi-LSTM model achieves a recall of 97.70%, indicating that it can effectively identify most malicious transaction behaviors before confirmation. This capability is particularly valuable for stablecoin systems because many DeFi attacks exploit transaction ordering, temporary liquidity, or abnormal interaction patterns that cannot be fully detected through static contract analysis alone. However, the current detection model still faces challenges in distinguishing sophisticated malicious transactions from legitimate high-frequency DeFi activities. The relatively lower precision compared with recall indicates that some normal transactions with unusual execution patterns may be incorrectly classified as suspicious behaviors. This limitation mainly originates from the highly imbalanced and dynamic characteristics of blockchain transaction environments. In practical deployment scenarios, the detection module may require adaptive threshold adjustment and continuous model updating to balance security sensitivity and operational efficiency.

From a practical deployment perspective, the proposed framework can be integrated into blockchain nodes, RPC gateways, or DeFi protocol monitoring systems. The low inference latency of the detection model demonstrates its potential for real-time transaction screening. However, further optimization is required to reduce computational overhead and ensure scalability in high-throughput blockchain environments.

6 Conclusion

In this paper, we propose a lifecycle-based dynamic security framework for stablecoin smart contracts, integrating preventive smart contract protection mechanisms with real-time transaction-level anomaly detection. The proposed framework addresses multiple attack surfaces, including contract execution vulnerabilities, oracle manipulation, and governance-related threats. Experimental results demonstrate that the proposed defense mechanisms can effectively mitigate representative stablecoin attacks, while the Bi-LSTM-based detection model achieves 96.61% accuracy and 97.70% recall with low inference latency, showing its potential for real-time blockchain security monitoring. Although the current evaluation is based on simulated transaction data, this work provides a practical foundation for combining intelligent detection with multi-layer security protection in decentralized financial systems.

Acknowledgments

We use AI translation software for grammar improvement and polishing.

References

[1] Akathoott, A. M., Rodriguez, A., and Burtscher, M. (2026). Sleek: Compressing memory copies for floating-point data on gpus. In *2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 115–129. IEEE.

[2] Akon, M., Toufikuzzaman, M., and Hussain, S. R. (2025). From control to chaos: A comprehensive formal analysis of 5g's access control. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1081–1100. IEEE.

[3] Aloudat, M. Z., Aboumadi, A., Soliman, A., Al-Mohammed, H., Al-Ali, M., Mahgoub, A., Barhamgi, M., and Yaacoub, E. (2025). Metaverse unbound: A survey on synergistic integration between semantic communication, 6g, and edge learning. *IEEE Access*.

[4] Alsunaidi, S. J., Aljamaan, H., and Hammoudeh, M. (2025). Leveraging machine learning models to improve smart contract security: A survey of vulnerabilities and detection methods. *ACM Computing Surveys*, 58(6):1–37.

[5] Baralla, G., Cocco, L., Di Francesco, M., and Tonelli, R. (2025). Integrating blockchain, ssi, and rbac for the secure management of defense heritage buildings. *IEEE Access*, 13:86290–86307.

[6] Chen, H., Chen, D., Yang, Y., Xu, L., Gao, L., Zhou, M., Hu, C., and Li, L. (2025a). Arkanalyzer: The static analysis framework for openharmony. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 136–147. IEEE.

[7] Chen, J., Shao, Z., Yang, S., Shen, Y., Wang, Y., Chen, T., Shan, Z., and Zheng, Z. (2025b). Numscout: Unveiling numerical defects in smart contracts using llm-pruning symbolic execution. *IEEE Transactions on Software Engineering*.

[8] Chen, S. (2025). A security analysis of web3. 0 cross-chain bridges. In *Proceedings of the 4th International Conference on Computer, Artificial Intelligence and Control Engineering*, pages 213–217.

[9] Fan, S. and Min, T. (2026). Web3agent: Automating on-chain operations via natural language interfaces. *ACM Transactions on the Web*, 20(1):1–27.

[10] Gao, B., Wang, Y., Wei, Q., Liu, Y., Goh, R. S. M., and Lo, D. (2025a). $airaclex$: Automated detection of price oracle manipulations via llm-driven knowledge mining and prompt generation. *IEEE Transactions on Services Computing*.

[11] Gao, P., Kong, D., and Li, X. (2025b). Implementation and security analysis of cryptocurrencies based on ethereum. *arXiv preprint arXiv:2504.21367*.

[12] Gomes, D., Felix, E., Aires, F., and Vieira, M. (2025). Static code analysis for iot security: A systematic literature review. *ACM Computing Surveys*, 58(3):1–47.

[13] Hossain, T., Hassan, S., Bappy, F. H., Yanhaona, M. N., Zaman, T. S., and Islam, T. (2025). Bridging immutability with flexibility: A scheme for secure and efficient smart contract upgrades. In *2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–5. IEEE.

[14] Hu, J. and Ruan, N. (2026). Blockchain attacks and defenses: A layered and cross-domain survey. *arXiv preprint arXiv:2607.06593*.

[15] Huang, Q., He, Y., Xing, Z., Yu, M., Xu, X., and Lu, Q. (2025). Enhancing fine-grained smart contract vulnerability detection through domain features and transparent interpretation. *IEEE Transactions on Reliability*, 74(3):4207–4221.

[16] Huang, X., Qiu, W., Jie, W., Xia, Q., Zhang, Q., Sun, Y., Xie, M., Liu, Y., and Zheng, Z. (2026). Flashshield: Detecting flash loan attacks in defi using hypergraph neural network. *IEEE Transactions on Dependable and Secure Computing*.

[17] Jarkas, O., Ko, R., Dong, N., and Mahmud, R. (2025). A container security survey: Exploits, attacks, and defenses. *ACM Computing Surveys*, 57(7):1–36.

[18] Kong, Z., Zhang, C., Xie, M., Hu, M., Xue, Y., Liu, Y., Wang, H., and Liu, Y. (2025). Smart contract fuzzing towards profitable vulnerabilities. *Proceedings of the ACM on Software Engineering*, 2(FSE):153–175.

[19] Lemayian, J. P., Gagnon, G., Zhang, K., and Giard, P. (2025). Evmx: An fpga-based accelerator for smart contract processing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 34(2):554–567.

[20] Li, L., Chen, Y., Wu, J., Pan, Y., and Yu, Z. (2025a). Understanding inconsistent state update vulnerabilities in smart contracts. *ACM Transactions on Software Engineering and Methodology*.

[21] Li, N., Qi, M., Xu, Z., Zhu, X., Zhou, W., Wen, S., and Xiang, Y. (2025b). Blockchain cross-chain bridge security: Challenges, solutions, and future outlook. *Distributed Ledger Technologies: Research and Practice*, 4(1):1–34.

[22] Li, W., Li, X., Mao, Y., and Zhang, Y. (2025c). Interaction-aware vulnerability detection in smart contract bytecodes. *IEEE Transactions on Dependable and Secure Computing*.

[23] Li, X., Li, W., Liu, Z., Zhang, Y., and Mao, Y. (2025d). Penetrating the hostile: Detecting defi protocol exploits through cross-contract analysis. *IEEE Transactions on Information Forensics and Security*, 20:11759–11774.

[24] Li, X., Li, Z., Li, W., and Zhang, Y. (2026a). A systematic survey of defi composability: From code correctness to protocol robustness. *Blockchain: Research and Applications*, page 100554.

[25] Li, X., Wang, X., Li, W., and Li, Z. (2026b). Psr^2: A phase-based semantic reasoning framework for atomicity violation detection via contract refinement. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, pages 1297–1301.

[26] Li, X., Xie, L., Li, W., and Li, Z. (2026c). Uscsa: Evolution-aware security analysis for proxy-based upgradeable smart contracts. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, pages 126–130.

[27] Li, X., Ye, S., Li, W., and Li, Z. (2026d). Scpatcher: Automated smart contract code repair via retrieval-augmented generation and knowledge graph. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, pages 1212–1216.

[28] Li, Y., Xiang, Y., Wang, Q., Yuen, T. H., Deppeler, A., and Yu, J. (2026e). Sok: Stablecoins in retail payments. In *2026 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–21. IEEE.

[29] Lin, B. and Lin, R. (2025). A parallel monetary system based on the redeemable self-decaying money–the ultimate hedge and safe haven of private wealth in the rising wave of over issuance of fiat and token money/stablecoin. *arXiv preprint arXiv:2511.00365*.

[30] Lin, R., Deng, X., Li, Q., Ma, J., Feng, Y., Qing, Y., Li, Z., Zhang, Y., Cui, S., Meng, C., et al. (2026). Safety in self-evolving llm agent systems: Threats, amplification, and case studies. *arXiv preprint arXiv:2606.23075*.

[31] Liu, D., Zhang, J., Wang, Y., Shen, H., Zhang, Z., and Ye, T. (2025a). Blockchain smart contract security: Threats and mitigation strategies in a lifecycle perspective. *ACM Computing Surveys*, 58(4):1–34.

[32] Liu, H., Wu, D., Sun, Y., Wang, S., and Liu, Y. (2025b). Have we solved access control vulnerability detection in smart contracts? a benchmark study. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1995–2007. IEEE.

[33] Liu, X., Yang, N., Li, C., Li, J., Ma, J., and Lu, K. (2026a). The dark side of flexibility: Detecting risky permission chaining attacks in serverless applications. In *NDSS*.

[34] Liu, Y., Xi, R., and Pattabiraman, K. (2025c). Reentrancy redux: The evolution of real-world reentrancy attacks on blockchains. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 576–588. IEEE.

[35] Liu, Z., Li, X., Li, W., and Li, Z. (2026b). Liquilm: Bridging the semantic gap in liquidity flaw audit via dcn and llms. *arXiv preprint arXiv:2604.03860*.

[36] Long, X., Wang, Y., and Li, X. (2025). From fomo3d to lottery dapp: Analysis of ethereum-based gambling applications. *arXiv preprint arXiv:2508.12303*.

[37] Lv, Z., Liu, X., Ma, L., Xu, H., and Li, K. (2026). Mvcx: An efficient multi-version-based concurrency control scheme for cross-chain smart contract transactions. In *IEEE INFOCOM 2026-IEEE Conference on Computer Communications*, pages 1–10. IEEE.

[38] Mahrous, A., Caprolu, M., and Di Pietro, R. (2025). Stablecoins: Fundamentals, emerging issues, and open challenges. *arXiv preprint arXiv:2507.13883*.

[39] Mishra, D. and Phansalkar, S. (2025). Blockchain security in focus: a comprehensive investigation into threats, smart contract security, cross-chain bridges, and vulnerabilities detection tools and techniques. *IEEE Access*, 13:60643–60671.

[40] Mo, J., Kuang, H., and Li, X. (2025). Password strength detection via machine learning: Analysis, modeling, and evaluation. *arXiv preprint arXiv:2505.16439*.

[41] Moghadam, E. E., Wyss, M., Kwon, J., Frei, M., Hu, Y.-C., Perrig, A., and Sonnino, A. (2026). Signet: Scalable network-driven proof of notification for blockchain systems. In *2026 IEEE 46th International Conference on Distributed Computing Systems (ICDCS)*, pages 502–512. IEEE.

[42] Moradi, J. and Hashemi, S. (2026). Smart contract vulnerabilities in ethereum: A systematic literature review and analytical synthesis. *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*, pages 1–29.

[43] Pathak, S., Vyas, V., and Malsa, N. (2025). Performance evaluation of erc-20 and erc-721 blockchain protocols through an optimization framework. In *2025 2nd International Conference on Advanced Computing and Emerging Technologies (ACET)*, pages 1–5. IEEE.

[44] Peng, C., Jiang, M., Zhou, Y., and Wu, L. (2026). Thought is all you need: Smart contract vulnerability detection with thought-augmented large language model. *Proceedings of the ACM on Software Engineering*, 3(FSE):3023–3046.

[45] Ren, S., He, L., Tu, T., Wu, D., Liu, J., Ren, K., and Chen, C. (2025). Lookahead: Preventing defi attacks via unveiling adversarial contracts. *Proceedings of the ACM on Software Engineering*, 2(FSE):1847–1869.

[46] Semwal, D., Ponnaganti, P. K., Dwivedi, V., Venkatesan, S., Shukla, S. K., et al. (2025). Security risk analysis of the blockchain application ecosystem. In *2025 Conference on Building a Secure & Empowered Cyberspace (BuildSEC)*, pages 108–117. IEEE.

[47] Shi, C., Li, Z., and Li, X. (2025). System password security: Attack and defense mechanisms. *arXiv preprint arXiv:2510.10246*.

[48] Sun, H., Wang, Y., and Li, X. (2025). From data behavior to code analysis: A multimodal study on security and privacy challenges in blockchain-based dapp. *arXiv preprint arXiv:2504.11860*.

[49] Tan, Z., Parambath, S. P., Anagnostopoulos, C., Singer, J., and Marnerides, A. K. (2025). Advanced persistent threats based on supply chain vulnerabilities: Challenges, solutions, and future directions. *IEEE Internet of Things Journal*, 12(6):6371–6395.

[50] Tiwari, V., Dasari, V. S. R., and Agrawal, K. (2025). Mitigating cryptocurrency misuse: A survey with a cross-domain adaptive framework. In *2025 IEEE 16th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 0427–0436. IEEE.

[51] Tong, F., Li, Z., Cheng, G., Zhang, Y., and Li, H. (2025). sbugchecker: A systematic framework for detecting solidity compiler-introduced bugs. *IEEE Transactions on Information Forensics and Security*.

[52] Wang, Y., He, R., Guo, H., Wang, H., Zhang, Y., and Wang, B. (2025). Prevention of flash loan attacking on the decentralized finance system of a public blockchain. In *International Conference of Pioneering Computer Scientists, Engineers and Educators*, pages 431–445. Springer.

[53] Wang, Y., Li, W., Li, X., Li, Z., Xie, L., and Zhang, Y. (2026a). Libscan: Smart contract library misuse detection with iterative feedback and static verification. *Blockchain: Research and Applications*, page 100521.

[54] Wang, Y., Yi, W., Li, W., Li, Z., and Li, X. (2026b). Ragas: Retrieval-augmented gas optimization for smart contracts with continuous knowledge integration. *arXiv preprint arXiv:2608.15857*.

[55] Wang, Z., Shen, Z., Chen, L., Song, W., Ge, J., Huang, L., and Luo, B. (2026c). Empirical analysis of smart contract factories on evm-compatible chains. *ACM Transactions on Software Engineering and Methodology*, 35(8):1–29.

[56] Wei, C., Cai, S., Charalambous, Y., Wu, T., Godboley, S., and Cordeiro, L. (2025). Veriexploit: Automatic bug reproduction in smart contracts via llms and formal methods. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1415–1426. IEEE.

[57] Wen, H., Li, S., Lau, R., and Zhang, J. (2025). Stablecoins and the emerging hybrid monetary ecosystems. *arXiv preprint arXiv:2505.10997*.

[58] Yan, J. (2025). On the necessity of stablecoins and the legal mechanisms for their regulation. In *Proceedings of the 2025 6th International Artificial Intelligence and Blockchain Conference*, pages 11–14.

[59] Yang, S., Chen, J., Xiao, L., Hu, J., Lin, D., Wu, J., Zhang, T., and Zheng, Z. (2025). Who is pulling the strings: Unveiling smart contract state manipulation attacks through state-aware dataflow analysis. *IEEE Transactions on Software Engineering*, 51(10):2942–2956.

[60] Ye, M., Nan, Y., Zhong, Z., Su, J., Lin, X., Zheng, P., and Zheng, Z. (2026). Chaindelta: Automatic patch-based exploit generation for ethereum with fuzzing agents. *Proceedings of the ACM on Software Engineering*, 3(FSE):3392–3414.

[61] You, S., Kuehlkamp, A., and Nabrzyski, J. (2025). Hybrid stabilization protocol for cross-chain digital assets using adaptor signatures and ai-driven arbitrage. In *International Conference on Financial Cryptography and Data Security*, pages 138–161. Springer.

[62] Zhang, B., He, N., Hu, X., Ma, K., and Wang, H. (2025a). Following devils' footprint: towards real-time detection of price manipulation attacks. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 4127–4145.

[63] Zhang, J., Zheng, Z., Nan, Y., Ye, M., Ning, K., Zhang, Y., and Zhang, W. (2025b). Smartreco: Detecting read-only reentrancy via fine-grained cross-dapp analysis. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2138–2150. IEEE.

[64] Zhang, L., Li, K., Sun, K., Wu, D., Liu, Y., Tian, H., and Liu, Y. (2025c). Acf ix: Guiding llms with mined common rbac practices for context-aware repair of access control vulnerabilities in smart contracts. *IEEE Transactions on Software Engineering*.

[65] Zhang, X., Zhu, Y., Li, X., Zhang, Y., Niu, W., Xu, F., He, J., Yan, R., and Huang, S. (2025d). Active cybersecurity: vision, model, and key technologies. *Frontiers of Information Technology & Electronic Engineering*, 26(8):1243–1278.

[66] Zhang, Z., Yin, W., and Li, X. (2025e). A novel gpt-based framework for anomaly detection in system logs. *arXiv preprint arXiv:2510.16044*.

[67] Zhou, W., Lyu, D., and Li, X. (2025). Blockchain security based on cryptography: a review. *arXiv preprint arXiv:2508.01280*.